TacHarness: Agentic Infrastructure for Continually Improving Visuotactile Robot Policies

Jingxiang Guo[1] **Gujie Shao**[1] **Yuxin Hu**[1] **Bingsheng Xie**[1] **Jiaxin Shi**[2] **Chen Gao**[1]
Yuhang Zheng[1] **Haoyu Zhao**[1] **Weihao Yuan**[3] **Fei Wang**[4] **Shuicheng Yan**[1]*
[1]National University of Singapore [2]Johns Hopkins University [3]Nanjing University [4]Alchedata

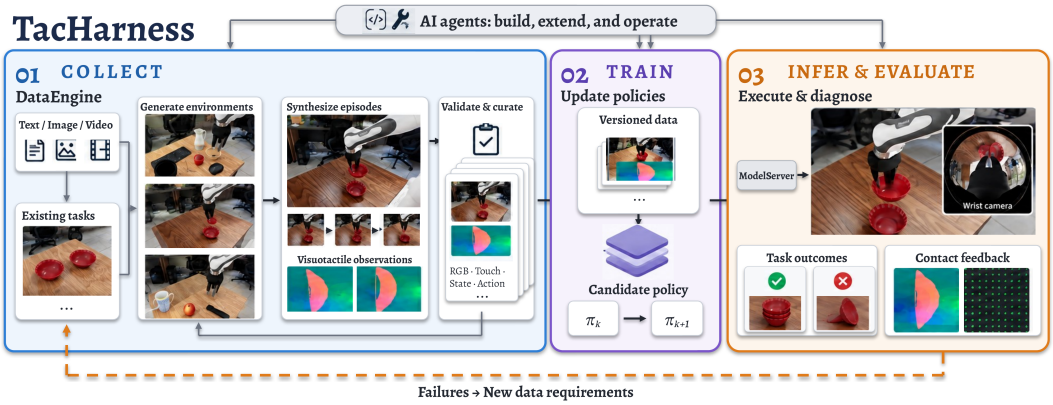


Figure 1: **TacHarness overview.** Agents coordinate a collection–training–evaluation pipeline through planning, tool invocation, validation, and repair. Environment generation and DataEngine produce validated multimodal data; training yields candidate policies; decoupled policy serving and benchmark execution return task outcomes and diagnostic evidence for subsequent decisions.

Abstract

Visuotactile robot learning involves environment construction, multimodal data production, policy training, and evaluation. TacHarness integrates agents into these stages to coordinate tool execution and manage dependencies between datasets and trained models. Agents interpret requirements, select tools and configurations, inspect execution evidence, and coordinate bounded retries or repairs. Collection integrates agentic environment generation with DataEngine. Environment-generation agents interpret text or visual requirements, select assets and layouts, invoke compilation and simulation, and diagnose and repair failures. After robot-task binding, DataEngine agents select trajectories, configure embodiment retargeting, and synthesize synchronized visual, tactile, state, and action observations through simulation. Quality checks connect accepted episodes to versioned datasets. For training, agents select policy backends, initialization, and optimization budgets, bind accepted datasets to compatible configurations, and invoke training executors. They inspect progress and failure evidence and coordinate recovery through backend checkpoint mechanisms. Training checks cover optimization steps, finite losses, and checkpoint availability; the resulting checkpoints record data sources and sensor–action conventions for downstream serving and evaluation. Evaluation decouples policy inference from benchmark execution and returns task outcomes, timing, and failure evidence for subsequent decisions. Shared formats link scenes, datasets, models, and evaluation reports, while deterministic tools enforce task-specific acceptance criteria. We present the system architecture and adapters alongside recorded agent execution and multimodal replay across six environments. Execution records connect data inspection and repair to policy training, checkpoint verification, and online benchmark evaluation, showing how agents coordinate tools and use results throughout the learning workflow.

*Corresponding author: yansc@nus.edu.sg

1 INTRODUCTION

Agentic systems in robotics use planning, tool calls, and execution feedback to carry out tasks (Li et al., 2026a;b). Their applications include generating tasks and demonstrations (Wang et al., 2023; Hua et al., 2024), organizing training and physical experiments (Li et al., 2026b; Xiao et al., 2026), connecting data collection, policy learning, and deployment (Li et al., 2026a), and assisting model and benchmark integration for evaluation (Choi et al., 2026; Robocurve, 2026). In each case an agent translates a requirement into jobs, inspects execution results, and selects the next operation.

This motivates an *agent-as-infrastructure* perspective: agents coordinate the tools and dependencies of a robot-learning workflow. Collection determines the data available for training; training produces a model with specific input and action conventions; evaluation executes that model and returns evidence for subsequent decisions. Connecting these stages requires interfaces that identify what each operation consumes, what it produces, and how completion is checked. Agents also need failure evidence that separates a configuration error from an unsuccessful robot task.

We study this infrastructure role in *visuotactile robot learning*. Tactile and force observations describe contact, deformation, and slip that may be difficult to infer from images alone (Shan et al., 2026). Using these observations introduces requirements throughout the learning workflow. Data generation must account for end-effector geometry and sensor placement, and align visual, tactile, state, and action streams. A generated scene also needs a robot task, demonstration source, and success criterion before it can supply training episodes. Training and serving must preserve the same sensor preprocessing and action representation, while evaluation must retain task outcomes and contact-related evidence. These requirements connect environment construction and data synthesis to policy training, model serving, and benchmark evaluation.

We introduce **TacHarness**, an agentic infrastructure that organizes this workflow into collection, training, and inference with evaluation (Figure 1). Collection combines agentic environment generation with DataEngine: agents construct and inspect scenes, bind tasks, configure end-effector trajectory retargeting, and invoke multimodal capture. Training agents select policy backends and configurations, launch optimization, inspect progress, and coordinate recovery. Serving and evaluation separate policy inference from benchmark execution and return task outcomes and failure evidence. Dataset manifests, model checkpoints, and reports connect the stages; simulators, training executors, and validators perform the operations and check their outputs.

Our contributions are as follows:

- **Agentic workflow orchestration.** We coordinate tools, data and model dependencies, validation, and recovery across visuotactile collection, training, and evaluation.
- **Agentic environment generation and multimodal data synthesis.** We connect scene generation and repair with task binding, end-effector retargeting, synchronized capture, and dataset validation and acceptance through DataEngine.
- **Training and evaluation interfaces.** We connect policy-specific training executors to model serving and environment adapters, illustrate replay across supported tasks, and define a protocol for assessing workflow execution and policy updates.

2 RELATED WORK

Visuotactile learning and benchmarks. Visual, tactile, and force observations support contact perception and manipulation (Shan et al., 2026). Existing approaches study multimodal representation learning, heterogeneous sensor adaptation, and tactile feedback for planning and action refinement (Sferrazza et al., 2024; Jones et al., 2025; Bi et al., 2025; Zhang et al., 2025). Taxim and TacSL provide tactile simulation and learning tools (Si & Yuan, 2021; Akinola et al., 2024). Manipulation benchmarks cover reusable tasks, diverse household scenes, bimanual demonstrations, and deformable objects (Liu et al., 2023; James et al., 2020; Nasiriany et al., 2024; Chen et al., 2025; 2022). Tactile benchmarks examine sensor choices, multimodal policies, force feedback, and deformation-aware success criteria (Luu et al., 2025; Zorin et al., 2026; Chen et al., 2026a; Wu et al., 2026; Jing et al., 2026). These platforms provide task definitions and sensor interfaces for visuotactile data collection and policy evaluation across contact-rich manipulation tasks.

Table 1: **Framework capabilities across robot learning.** ✓: documented; ✗: not reported (September 2026). Agentic columns cover LLM/VLM-driven environment generation, collection, training configuration and launch, and benchmark adaptation and rollout checks.

Framework	Tactile	Agentic workflow			
		Env. generation	Data collection	Policy training	Benchmark integration
TacSL (2024)	✓	✗	✗	✗	✗
ManiFeel (2025)	✓	✗	✗	✗	✗
UniVTAC (2026a)	✓	✗	✗	✗	✗
IsaacLab (2026)	✓	✓	✗	✓	✓
GenSim (2024)	✗	✓	✓	✗	✗
RoboGen (2023)	✗	✓	✓	✓	✗
GenSim2 (2024)	✗	✓	✓	✓	✗
EmbodiedGen V2 (2026)	✗	✓	✗	✗	✗
glm-skills-embodied (2026)	✗	✓	✓	✗	✗
Eureka (2023)	✗	✗	✗	✓	✗
DrEureka (2024)	✗	✗	✗	✓	✗
Eurekaverse (2024)	✗	✓	✗	✓	✗
RoboClaw (2026a)	✗	✗	✓	✓	✗
ENPIRE (2026)	✗	✗	✓	✓	✗
HARBOR (2026b)	✗	✓	✗	✓	✓
autoresearch-robotics (2026)	✗	✗	✗	✓	✗
vla-eval (2026)	✗	✗	✗	✗	✓
TacHarness (ours)	✓	✓	✓	✓	✓

Agents for data production and training. RoboGen, GenSim2, and glm-skills-embodied generate simulation tasks, expert programs, and demonstrations (Wang et al., 2023; Hua et al., 2024; hxwxss, 2026). EmbodiedGen constructs simulation assets and worlds, with V2 exposing stateful editing and validation through agent-callable skills (Wang et al., 2025; 2026). Agents also optimize rewards, randomization, and training curricula (Ma et al., 2023; 2024; Liang et al., 2024), and conduct policy experiments through training-code revisions and feedback from repeated simulation and physical robot experiments (Xiao et al., 2026; Li et al., 2026b; jellyheadandrew, 2026).

Workflow and evaluation infrastructure. RoboClaw connects collection, policy learning, and deployment (Li et al., 2026a). XPolicyLab standardizes observation and action interfaces, isolates policy dependencies, and provides agent skills for adapters (XPolicyLab Community et al., 2026). The vla-eval harness supports agent-assisted integration of policy servers and benchmarks (Choi et al., 2026). AutoEval schedules real-robot evaluation jobs with automatic resets and success detection (Zhou et al., 2025); RobotArena ∞ reconstructs evaluation scenes from demonstrations and combines VLM scoring with human preferences (Jangir et al., 2026). Inspect Robots supplies policy interfaces and scored execution records (Robocurve, 2026). Table 1 compares frameworks by tactile support and agent involvement in environment generation, data collection, policy training, and benchmark integration. TacHarness connects agent-operated collection, training, and evaluation through shared interfaces for synchronized visuotactile datasets, trained models, and evaluation results.

3 TacHarness

3.1 System Overview and Agent Roles

Figure 2 connects collection, training, and evaluation. Accepted episodes enter a dataset; training produces a checkpoint; evaluation informs policy selection and subsequent collection. Each stage records its inputs, outputs, outcomes, and execution settings, allowing agents to select subsequent jobs using the validated scenes, datasets, and checkpoints from earlier stages.

Agents as an orchestration layer. Agents translate requirements into plans, select providers, and manage dependencies. Skills expose tools; adapters define their observation, action, and output

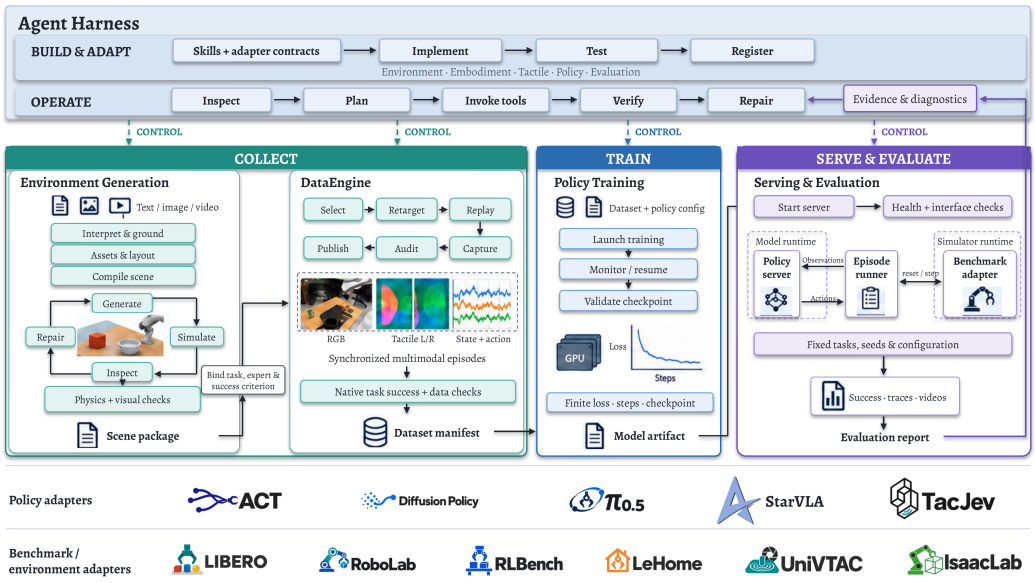


Figure 2: **TacHarness architecture.** Agents orchestrate tools across collection, training, and evaluation, using shared interfaces to inspect results and coordinate repairs. Validated scenes, dataset manifests, model checkpoints, and evaluation reports connect the stages. Policy serving is decoupled from benchmark execution. Bottom rows show the policy and environment adapters.

interfaces. The build-and-adapt path implements missing interfaces, checks their inputs and outputs, and registers providers for reuse. The operating path invokes tools and checks results.

Invoke, verify, and repair. Agents inspect tool results and propose bounded retries or repairs. Simulators, dataset validators, and task criteria check the resulting outputs. DataEngine manages episode production and publication, training providers perform optimization, and ModelServer handles inference. Dataset manifests, checkpoints, and evaluation reports connect these stages.

Policy and environment adapters. The policy row in Figure 2 includes ACT (Zhao et al., 2023), Diffusion Policy (Chi et al., 2023), $\pi_{0.5}$ (Physical Intelligence et al., 2025), StarVLA (StarVLA Community, 2026), and TacJev, our multimodal adaptation drawing on Jev (Almeida, 2026) and JEMM (MaestroYan, 2026). The environment row comprises LIBERO (Liu et al., 2023), RoboLab (Yang et al., 2026), RLBench (James et al., 2020), LeHome (Li et al., 2026c), UniVTAC (Chen et al., 2026a), and IsaacLab (Mittal et al., 2025). These adapters map source-specific observations, actions, and task interfaces into the collection and evaluation stages of the shared workflow.

3.2 Environment Generation

Construct scenes from requirements. Agents construct scenes through three routes (Figure 3): Text2Env uses object and relation descriptions, Anchor2Env adds visual anchors, and media reconstruction supplies geometry and poses. All routes share asset resolution and scene construction while preserving input-specific constraints. Assets come from catalogs, acquisition, or reconstruction. Their records retain sources, dimensions, coordinate systems, scale conventions, physical properties, and backend-specific representations and checks that guide simulator adapters in selecting usable assets and constructing scenes for the target backend.

Validate, repair, and qualify tasks. A VLM inspects rendered views for object placements inconsistent with everyday use. In Figure 3, this judgment prompts the agent to correct object poses. Simulation checks then verify spatial relations, contact, penetration, containment, and settling. Task qualification binds the robot, reset procedure, expert or trajectory, action semantics, and native success and termination criteria. After interaction checks, DataEngine receives the scene, task specification, source trajectory, and validation results for multimodal collection.

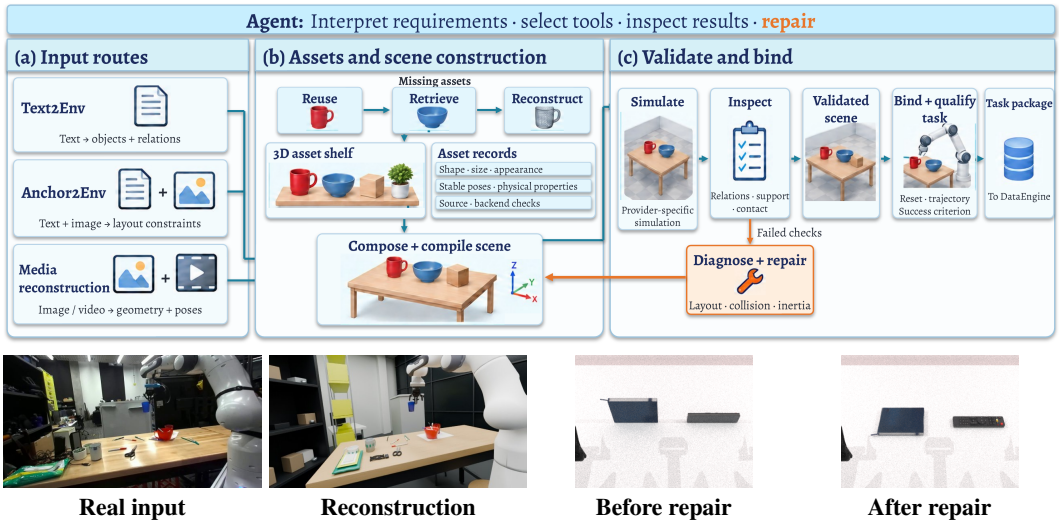
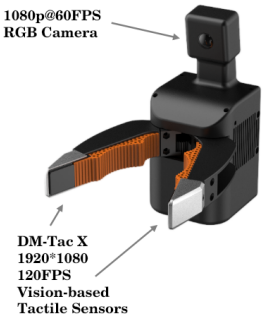


Figure 3: **Agentic environment generation.** Three input routes feed asset resolution and scene construction. Asset records retain sources and backend checks. Agents inspect scenes and invoke repairs; simulation checks and task qualification prepare environments for DataEngine collection.



From embodiment to training data

1. Bind. Register the gripper geometry, camera calibration, and positions of the tactile sensing surfaces.

2. Retarget. Convert source actions into target end-effector poses and gripper opening commands.

3. Replay and capture. Record RGB, tactile observations, state, and actions on a shared episode timeline.

4. Validate and save. Retain task outcomes, modality checks, and source information with the published dataset.

Figure 4: **DM-TacClaw and its collection interface.** The end-effector combines an RGB camera and two vision-based tactile sensing surfaces. Hardware specifications are annotated beside the device; simulation streams use the configured recording rate.

3.3 DataEngine

DataEngine converts task, embodiment, sensor, variation, and budget requirements into collection jobs. Agents obtain source trajectories through dataset retrieval or trajectory generation, then configure retargeting and multimodal replay.

Retrieve or generate source trajectories. For existing environments, the agent searches online robot datasets and downloads trajectories matching the task and robot. It checks action representations, coordinate frames, and recording rates, then retargets recorded joint motions, end-effector poses, and gripper commands. Simulation feedback guides trajectory adjustments for the target embodiment and scene. When suitable demonstrations are unavailable, the agent generates motions step by step, inspects simulation outcomes, and revises the trajectory. Both routes retain source information and check task success before multimodal collection.

End-effector retargeting. End-effector adapters bind gripper geometry, contact surfaces, camera calibration, and mapping from actions to gripper opening to the source robot. DM-TacClaw provides one such configuration, combining an RGB camera and vision-based tactile sensors on opposing fingertips (Figure 4). The agent selects a source task and trajectory and configures retargeting; deterministic replay tools execute the mapped motion and render the resulting observations. Bimanual configurations maintain camera and tactile-pad identities for each arm.

Multimodal capture. Retargeted replay produces synchronized RGB, contact-conditioned tactile observations, proprioceptive state, and actions. Episode timestamps, calibration, embodiment identity, and action units and frames preserve interpretation across training and deployment; replay supports inspection of contact onset, gripper motion, and object interaction.

Tactile simulation. Tactile streams come from two sensor models chosen by the environment adapter: the GelSight Mini pads of LIBERO/Tabero and UniVTAC follow the TacEx sensor model with Taxim (Si & Yuan, 2021) or UIPC-based rendering, and the DM-TacClaw pads of the other replays use TacSim depth-camera indentation gated by PhysX contact. Appendix B specifies both pipelines, their calibration status, and example streams.

Validate and publish. Demonstrations pass checks for native task success, complete sensor streams, source information, and scene and state replay. Jobs retain configurations, trajectory identities, outcomes, and media, including failures for diagnosis and learning. Accepted episodes are exported to standard formats with a versioned manifest recording scenes, sensors, and validation results. The browser supports queue monitoring and synchronized replay. Aligned images and robot states help identify failed interactions and trajectory segments for revision.

3.4 TRAINING

The agent selects a policy backend, registered dataset manifest, initialization, and optimization budget, then invokes the training executor. The selected backend defines the architecture and training objective; the job record links the input dataset and configuration to the resulting model.

Tactile inputs and policy adaptation. ACT, Diffusion Policy, $\pi_{0.5}$, and StarVLA support two input routes. The image route adds each tactile image as an additional camera view processed by the policy's visual encoder. The dedicated route uses a pretrained tactile masked autoencoder, learnable query tokens, and a linear projection to match the policy feature dimension. Each sensor produces eight tokens by default; their ordering follows the configured sensor list. Training updates the encoder jointly with the policy or freezes its backbone, learning only queries and projection.

ACT places tactile tokens in the observation encoder or appends them to the action decoder's attention memory. Diffusion Policy flattens tactile tokens for each observation step and concatenates them with visual and state features to condition action denoising. For $\pi_{0.5}$, tactile tokens join the image–language prefix or form a separate conditioning block visible to the action expert. StarVLA–GR00T appends them to the VLM features consumed by the action head; compatible backbones also support insertion into the VLM input sequence. Both routes use the policy backend's training objective.

TacJev: tactile-conditioned meta-action decisions. TacJev adapts a multimodal backbone to predict the next meta-action from RGB images, left and right tactile images, the task instruction, and end-effector state. Sensor names identify each image stream. Inspired by Jev's structured decisions (Almeida, 2026) and JEMM's candidate scoring (MaestroYan, 2026), the model maps each candidate to a single token and scores all candidates in one forward pass. The default vocabulary follows Show-Harness's single-arm interface (Chen et al., 2026b): six translation directions, grasp, release, and completion. An embodiment-specific interpreter supplies the reference frame, step size, workspace bounds, and physical gripper opening for each decision.

Supervised fine-tuning first minimizes cross-entropy over the allowed candidates. A second stage trains next-action probabilities with $\mathcal{L} = -\log p_y + \lambda \sum_{a \in \mathcal{A}} (p_a - \mathbf{1}[a = y])^2$, where y is the expert meta-action and $\mathcal{A}$ is the allowed action set. Episode-disjoint calibration data determines a positive softmax temperature, while separate test episodes measure action accuracy and probability calibration. During execution, fresh visual and tactile observations condition each decision, and the benchmark's native task predicate supplies the evaluation outcome.

Launch, monitor, and recover. Training handlers bind datasets and configurations, launch jobs, and check optimization steps, finite losses, and checkpoints. Recovery restores model and optimizer state and resumes data processing. Checkpoints retain preprocessing, sensor ordering, synchronization, action normalization, and missing-modality settings for serving. Each run records its dataset split and budget, linking policy updates to new and earlier episodes.

3.5 Inference and Evaluation

Evaluation separates policy inference, rollout control, and benchmark simulation. Shared interfaces carry observations and actions, while benchmark adapters retain native task success criteria.

Launch a policy service. The agent launches the selected checkpoint with a serving configuration and checks readiness. ModelServer handles multimodal preprocessing, session state, and action prediction through a shared client interface. Before execution, the runner checks timestamps, tactile freshness, calibration, control rate, action units and frames, and validity intervals. Bounded queues, overload rejection, session isolation, and action-value checks govern execution.

Bind and run evaluation. Following the model–benchmark separation of the VLA Evaluation Harness (Choi et al., 2026), TacHarness connects a *policy server*, *evaluation runner*, and *benchmark adapter*. The agent selects the checkpoint, tasks, seeds, and evaluation configuration. After compatibility and health checks, the runner iterates observation acquisition, policy prediction, and environment stepping, retaining episode traces and outcomes. The server owns model loading and multimodal preprocessing; the runner owns episode scheduling and recording; the adapter exposes reset, observations, action execution, and native task outcomes. Policy and simulator runtimes retain separate dependencies. Adding a benchmark therefore focuses on its observation and action mappings, reset behavior, and success checks. The agent inspects these interfaces, implements missing mappings, and verifies a short rollout before running the configured evaluation.

Reports retain tasks, episode counts, reset and termination rules, native success metrics, and failure categories. Diagnostics track latency, deadline misses, stale inputs, and recovery.

Return evidence for the next decision. Synchronized replay and execution records help the agent diagnose scene, sensor, policy, and timing failures. It can request additional data or repair the affected adapter and rerun execution checks. Candidate acceptance uses task performance and regression checks on earlier tasks to guide subsequent collection and training.

4 System Demonstrations

4.1 Agent-Operated Training and Evaluation

A language-model agent prepared, trained, and evaluated a visuotactile policy on LIBERO's *put the bowl on the stove* task using one accepted Panda demonstration: 104 frames with two RGB and two GS Mini tactile views. Appendix A records the agent's decisions, tool results, and execution settings.

Data inspection identified a timing mismatch between actions and observations: observations were recorded before each step, while action fields stored the resulting end-effector state. The agent checked the recorder, corrected the comparison to the following observation, and restarted the job with the source data and numerical tolerance unchanged.

The agent trained ACT for 200 steps on one RTX 5090 and reloaded the checkpoint for evaluation. It adapted absolute end-effector targets and gripper opening to simulator IK and gripper controls, then evaluated the policy for 300 actions on each of three reset seeds. The initial policy achieved 0/3 task successes. Traces and videos retain predicted actions and simulator task signals. Training took 28.3 seconds, with a final loss of 2.081. Checkpoint loading checked parameter names and tensor shapes before running the policy on the three evaluation seeds. Each step used fresh RGB, tactile, and state observations; the runner executed the first action of the predicted chunk and required the native success predicate to hold for ten consecutive steps. Subsequent fine-tuning increased task success to 3/3 using the same three reset seeds and the same 300-step episode limit. The same seeds served to diagnose the initial failure, so this is one closed repair cycle rather than evidence of generalization.

4.2 Temporal Replay Across Environments

Recorded demonstrations in Figure 5 illustrate the collection and replay interface across six environments, including bimanual manipulation in LeHome. The sequences cover placement, grasping, garment folding, and stacking, showing robot motion and object interaction over time.

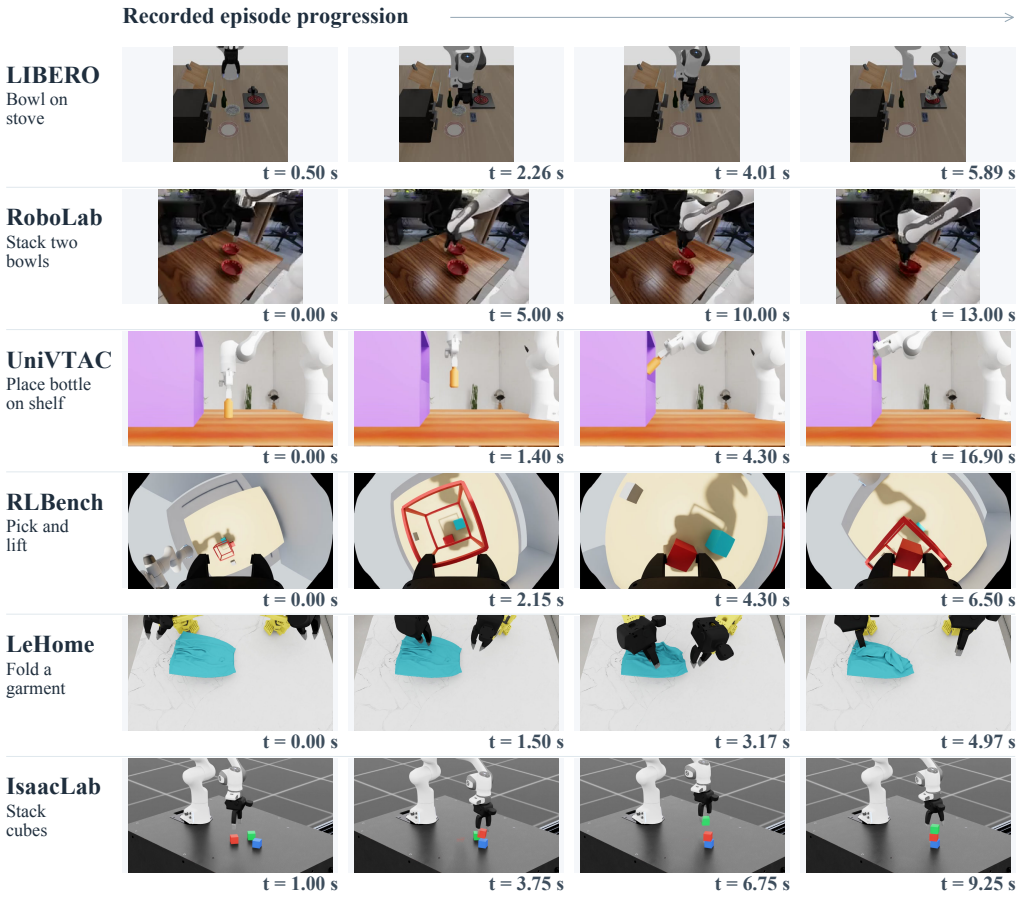


Figure 5: **Recorded episodes across six environments.** Four keyframes per task show motion and object interaction on independent time axes; LeHome uses a bimanual configuration.

4.3 Recorded Evidence and Limitations

Appendix B shows the tactile streams delivered to training (Figure 6). Three limits follow from the recorded runs: agent reliability is reported per run, with no scripted baseline under a matched budget; no experiment isolates the contribution of tactile input; and the collect–train–evaluate loop has been closed once, on one demonstration, so the continual improvement in the title is the target of the infrastructure, not a demonstrated result. A matched-budget comparison with a scripted workflow, covering held-out performance, earlier-task retention, recovery, runtime, human intervention, and contact quality (Jing et al., 2026), is the planned next measurement.

5 Conclusion

TacHarness integrates agents into visuotactile robot learning infrastructure. Agents coordinate environment generation, trajectory retrieval and refinement, multimodal collection, policy training, and benchmark evaluation through shared tools and interfaces. Scene validation and end-effector retargeting connect environments and demonstrations to synchronized visual and tactile data; training and serving preserve sensor and action conventions; evaluation returns task outcomes and diagnostic evidence for the next decision.

By linking these stages, TacHarness makes datasets, trained policies, and execution results reusable across successive learning cycles. Its adapters connect different embodiments and policy backends through shared tools, and recorded outcomes guide the next cycle of collection and policy updates.

REFERENCES

Iretiayo Akinola, Jie Xu, Jan Carius, Dieter Fox, and Yashraj Narang. TacSL: A Library for Visuotactile Sensor Simulation and Learning. *arXiv preprint arXiv:2408.06506*, 2024. URL https://arxiv.org/abs/2408.06506.

Diogo Almeida. Introducing system one models & Jev. TypeSafe AI. https://typesafe.ai/blog/introducing-system-one-models-and-jev, 2026. September 15, 2026.

Jianxin Bi, Kevin Yuchen Ma, Ce Hao, Mike Zheng Shou, and Harold Soh. VLA-Touch: Enhancing Vision-Language-Action Models with Dual-Level Tactile Feedback. *arXiv preprint arXiv:2507.17294*, 2025. URL https://arxiv.org/abs/2507.17294.

Baijun Chen, Weijie Wan, Tianxing Chen, Xianda Guo, Congsheng Xu, Yuanyang Qi, Haojie Zhang, Longyan Wu, Tianling Xu, Zixuan Li, et al. UniVTAC: A unified simulation platform for visuo-tactile manipulation data generation, learning, and benchmarking. *arXiv preprint arXiv:2602.10093*, 2026a. URL https://arxiv.org/abs/2602.10093.

Siwei Chen, Yiqing Xu, Cunjun Yu, Linfeng Li, Xiao Ma, Zhongwen Xu, and David Hsu. DaXBench: Benchmarking Deformable Object Manipulation with Differentiable Physics. *arXiv preprint arXiv:2210.13066*, 2022. URL https://arxiv.org/abs/2210.13066.

Tianxing Chen, Zanxin Chen, Baijun Chen, Zijian Cai, Yibin Liu, Zixuan Li, Qiwei Liang, Xianliang Lin, Yiheng Ge, Zhenyu Gu, Weiliang Deng, Yubin Guo, Tian Nian, Xuanbing Xie, Qiangyu Chen, Kailun Su, Tianling Xu, Guodong Liu, Mengkang Hu, Huan-ang Gao, Kaixuan Wang, Zhixuan Liang, Yusen Qin, Xiaokang Yang, Ping Luo, and Yao Mu. RoboTwin 2.0: A scalable data generator and benchmark with strong domain randomization for robust bimanual robotic manipulation. *arXiv preprint arXiv:2506.18088*, 2025. URL https://arxiv.org/abs/2506.18088.

Yanzhe Chen, Zechen Bai, Zhijun Cao, Wenzheng Zeng, Kevin Qinghong Lin, Yiqi Lin, Guoqiang Liang, Kevin Yuchen Ma, Qiming Huang, and Mike Zheng Shou. Show-Harness: Just a VLM agent can play robots, 2026b. URL https://arxiv.org/abs/2609.10522.

Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *arXiv preprint arXiv:2303.04137*, 2023. URL https://arxiv.org/abs/2303.04137.

Suhwan Choi, Yunsung Lee, Yubeen Park, Chris Dongjoo Kim, Ranjay Krishna, Dieter Fox, and Youngjae Yu. vla-eval: A unified evaluation harness for vision-language-action models. *arXiv preprint arXiv:2603.13966*, 2026. URL https://arxiv.org/abs/2603.13966.

Pu Hua, Minghuan Liu, Annabella Macaluso, Yunfeng Lin, Weinan Zhang, Huazhe Xu, and Lirui Wang. GenSim2: Scaling Robot Data Generation with Multi-modal and Reasoning LLMs. *arXiv preprint arXiv:2410.03645*, 2024. URL https://arxiv.org/abs/2410.03645.

hxwxss. GLM-5.3-Flash Skills: Embodied Data Engine. https://github.com/hxwxss/glm-skills-embodied, 2026. Software repository. Accessed September 28, 2026.

Isaac Lab Contributors. Isaac Lab: Agent skills and simulation framework. https://github.com/isaac-sim/IsaacLab, 2026. Official repository, revision d11f811. Accessed September 28, 2026.

Stephen James, Zicong Ma, David Rovick Arrojo, and Andrew J. Davison. RLBench: The robot learning benchmark & learning environment. *IEEE Robotics and Automation Letters*, 2020. URL https://arxiv.org/abs/1909.12271.

Yash Jangir, Yidi Zhang, Kashu Yamazaki, Chenyu Zhang, Kuan-Hsun Tu, Tsung-Wei Ke, Lei Ke, Yonatan Bisk, and Katerina Fragkiadaki. RobotArena ∞: Scalable robot benchmarking via real-to-sim translation. In *International Conference on Learning Representations*, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/2aa3da3c1463ee2cdaaee94be4f8ba3f-Abstract-Conference.html.

jellyheadandrew. autoresearch-robotics: Autonomous robotics research with simulation feedback. https://github.com/jellyheadandrew/autoresearch-robotics, 2026. Software repository. Accessed September 28, 2026.

Bowen Jing, Mingxin Wang, Ruiyang Hao, Chenchen Ge, Hanwen Shen, Junjie He, Yang Cui, Yiming Hou, Weitao Zhou, Jiawei Wang, et al. SoftVTBench: A Safety-Aware Visuo-Tactile Benchmark for Physically Constrained Robotic Manipulation of Deformable Objects (Early Version). *arXiv preprint arXiv:2607.04234*, 2026. URL https://arxiv.org/abs/2607.04234.

Joshua Jones, Oier Mees, Carmelo Sferrazza, Kyle Stachowicz, Pieter Abbeel, and Sergey Levine. Beyond sight: Finetuning generalist robot policies with heterogeneous sensors via language grounding. In *IEEE International Conference on Robotics and Automation (ICRA)*, pp. 5961–5968, 2025. URL https://arxiv.org/abs/2501.04693.

Ruiying Li, Yunlang Zhou, YuYao Zhu, Kylin Chen, Jingyuan Wang, Sukai Wang, Kongtao Hu, Minhui Yu, Bowen Jiang, Zhan Su, et al. RoboClaw: An Agentic Framework for Scalable Long-Horizon Robotic Tasks. *arXiv preprint arXiv:2603.11558*, 2026a. URL https://arxiv.org/abs/2603.11558.

Zechu Li, Yufeng Jin, Xiaoyang Liu, Puze Liu, Vignesh Prasad, Carlo D'Eramo, and Georgia Chalvatzaki. HARBOR: A Harness Framework for Agentic Robot Reinforcement Learning. *arXiv preprint arXiv:2606.08610*, 2026b. URL https://arxiv.org/abs/2606.08610.

Zeyi Li, Yushi Yang, Shawn Xie, Kyle Xu, Tianxing Chen, Yuran Wang, Zhenhao Shen, Yan Shen, Yue Chen, Wenjun Li, Yukun Zheng, Chaorui Zhang, Siyi Lin, Fei Teng, Hongjun Yang, Ming Chen, Steve Xie, and Ruihai Wu. LeHome: A simulation environment for deformable object manipulation in household scenarios. *arXiv preprint arXiv:2604.22363*, 2026c. URL https://arxiv.org/abs/2604.22363.

William Liang, Sam Wang, Hung-Ju Wang, Osbert Bastani, Dinesh Jayaraman, and Yecheng Jason Ma. Eurekaverse: Environment Curriculum Generation via Large Language Models. *arXiv preprint arXiv:2411.01775*, 2024. URL https://arxiv.org/abs/2411.01775.

Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. *arXiv preprint arXiv:2306.03310*, 2023. URL https://arxiv.org/abs/2306.03310.

Quan Khanh Luu, Pokuang Zhou, Zhengtong Xu, Zhiyuan Zhang, Qiang Qiu, and Yu She. ManiFeel: Benchmarking and Understanding Visuotactile Manipulation Policy Learning. *arXiv preprint arXiv:2505.18472*, 2025. URL https://arxiv.org/abs/2505.18472.

Yecheng Jason Ma, William Liang, Guanzhi Wang, De-An Huang, Osbert Bastani, Dinesh Jayaraman, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Eureka: Human-Level Reward Design via Coding Large Language Models. *arXiv preprint arXiv:2310.12931*, 2023. URL https://arxiv.org/abs/2310.12931.

Yecheng Jason Ma, William Liang, Hung-Ju Wang, Sam Wang, Yuke Zhu, Linxi Fan, Osbert Bastani, and Dinesh Jayaraman. DrEureka: Language Model Guided Sim-To-Real Transfer. *arXiv preprint arXiv:2406.01967*, 2024. URL https://arxiv.org/abs/2406.01967.

MaestroYan. JEMM: Multimodal structured decision model. https://huggingface.co/MaestroYan/JEMM, 2026. Model card. Accessed September 28, 2026.

Mayank Mittal, Pascal Roth, James Tigue, Antoine Richard, Octi Zhang, Peter Du, et al. Isaac Lab: A gpu-accelerated simulation framework for multi-modal robot learning. *arXiv preprint arXiv:2511.04831*, 2025. URL https://arxiv.org/abs/2511.04831.

Soroush Nasiriany, Abhiram Maddukuri, Lance Zhang, Adeet Parikh, Aaron Lo, Abhishek Joshi, Ajay Mandlekar, and Yuke Zhu. RoboCasa: Large-Scale Simulation of Everyday Tasks for Generalist Robots. *arXiv preprint arXiv:2406.02523*, 2024. URL https://arxiv.org/abs/2406.02523.

Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, et al. $\pi_{0.5}$: A vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025. URL https://arxiv.org/abs/2504.16054.

Robocurve. Inspect Robots: The Open-Source Evaluation Framework for Physical AI. https://github.com/robocurve/inspect-robots, 2026. Software repository. Accessed September 28, 2026.

Carmelo Sferrazza, Younggyo Seo, Hao Liu, Youngwoon Lee, and Pieter Abbeel. The power of the senses: Generalizable manipulation from vision and touch through masked multimodal learning. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 9698–9705, 2024. URL https://arxiv.org/abs/2311.00924.

Shilin Shan, Chuhao Zhou, Ruize Wang, Xinyan Chen, Xiangyu Chen, Xinyu Zhou, Boyu Ma, Iris Yuxuan Hu, Jingliang Li, Celeste Yuxuan Hu, et al. Learning Physical Interaction: A Survey of Tactile- and Force-aware Robot Learning. *arXiv preprint arXiv:2608.07558*, 2026. URL https://arxiv.org/abs/2608.07558.

Zilin Si and Wenzhen Yuan. Taxim: An Example-based Simulation Model for GelSight Tactile Sensors. *arXiv preprint arXiv:2109.04027*, 2021. URL https://arxiv.org/abs/2109.04027.

StarVLA Community. StarVLA: A lego-like codebase for vision-language-action model developing. *arXiv preprint arXiv:2604.05014*, 2026. URL https://arxiv.org/abs/2604.05014.

Lirui Wang, Yiyang Ling, Zhecheng Yuan, Mohit Shridhar, Chen Bao, Yuzhe Qin, Bailin Wang, Huazhe Xu, and Xiaolong Wang. GenSim: Generating Robotic Simulation Tasks via Large Language Models. In *International Conference on Learning Representations*, 2024. URL https://arxiv.org/abs/2310.01361.

Xinjie Wang, Liu Liu, Yu Cao, Ruiqi Wu, Wenkang Qin, Dehui Wang, Wei Sui, and Zhizhong Su. EmbodiedGen: Towards a Generative 3D World Engine for Embodied Intelligence. *arXiv preprint arXiv:2506.10600*, 2025. URL https://arxiv.org/abs/2506.10600.

Xinjie Wang, Liu Liu, Taojun Ding, Andrew Choi, Chaodong Huang, Mengao Zhao, Ziang Li, Jackson Jiang, Chunlei Yu, Shengxiang Liu, Wei Xu, and Zhizhong Su. EmbodiedGen V2: An Agentic, Simulation-Ready 3D World Engine for Embodied AI. *arXiv preprint arXiv:2607.07459*, 2026. URL https://arxiv.org/abs/2607.07459.

Yufei Wang, Zhou Xian, Feng Chen, Tsun-Hsuan Wang, Yian Wang, Katerina Fragkiadaki, Zackory Erickson, David Held, and Chuang Gan. RoboGen: Towards Unleashing Infinite Data for Automated Robot Learning via Generative Simulation. *arXiv preprint arXiv:2311.01455*, 2023. URL https://arxiv.org/abs/2311.01455.

Qiwei Wu, Rui Zhang, Xin Xiang, Tao Li, Weihua Zhang, Junjie Lai, and Renjing Xu. Tabero: Learning Gentle Manipulation with Closed-Loop Force Feedback from Vision, Touch, and Language. *arXiv preprint arXiv:2605.27886*, 2026. URL https://arxiv.org/abs/2605.27886.

Wenli Xiao, Jia Xie, Tonghe Zhang, Haotian Lin, Letian "Max" Fu, Haoru Xue, Jalen Lu, Yi Yang, Cunxi Dai, Zi Wang, et al. ENPIRE: Agentic Robot Policy Self-Improvement in the Real World. *arXiv preprint arXiv:2606.19980*, 2026. URL https://arxiv.org/abs/2606.19980.

XPolicyLab Community, Tianxing Chen, Yue Chen, Tian Nian, Zijian Cai, Guangyu Chen, Wenwei Lin, Qiwei Liang, Peicheng Xiang, Kailun Su, et al. XPolicyLab: A unified standard and open ecosystem for robot policy evaluation and deployment. *arXiv preprint arXiv:2608.09892*, 2026. URL https://arxiv.org/abs/2608.09892.

Xuning Yang, Rishit Dagli, Alex Zook, Hugo Hadfield, Ankit Goyal, Stan Birchfield, Fabio Ramos, and Jonathan Tremblay. RoboLab: A high-fidelity simulation benchmark for analysis of task generalist policies. In *Robotics: Science and Systems*, 2026. URL https://arxiv.org/abs/2604.09860.

Chaofan Zhang, Peng Hao, Xiaoge Cao, Xiaoshuai Hao, Shaowei Cui, and Shuo Wang. VTLA: Vision-Tactile-Language-Action Model with Preference Learning for Insertion Manipulation. *arXiv preprint arXiv:2505.09577*, 2025. URL https://arxiv.org/abs/2505.09577.

Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. *arXiv preprint arXiv:2304.13705*, 2023. URL https://arxiv.org/abs/2304.13705.

Zhiyuan Zhou, Pranav Atreya, You Liang Tan, Karl Pertsch, and Sergey Levine. AutoEval: Autonomous Evaluation of Generalist Robot Manipulation Policies in the Real World. *arXiv preprint arXiv:2503.24278*, 2025. URL https://arxiv.org/abs/2503.24278.

Anya Zorin, Zilin Si, Myungsun Park, Junsung Park, Alexiy Buynitsky, Sachin Bhadang, Taejun Park, Sohee John Yoon, Yong-Lae Park, Oliver Kroemer, et al. TacO: Benchmarking Tactile Sensors for Object Manipulation. *arXiv preprint arXiv:2605.21976*, 2026. URL https://arxiv.org/abs/2605.21976.

A Recorded Agent Execution

This appendix provides the configuration and execution details for the initial agent run in Section 4. Timestamped tool calls retain the selected settings, commands, returned outputs, and checks used to connect data preparation, training, and evaluation in the reported simulation run.

Table 2: **Agent execution and policy evaluation.** Data preparation, initial ACT training, and LIBERO evaluation before and after fine-tuning with matched reset seeds and episode limits.

Request	Agent decision and tool	Observed result
Prepare data	Inspect the provider and accepted data; repair module discovery and the pre/post-step check	1 existing episode, 104 frames; two RGB and two tactile views
Train ACT	Configure and launch training; inspect loss; reload the saved checkpoint	200 steps in 28.3 s; final training loss 2.081; strict reload passed
Initial evaluation	Add observation and IK adapters; load the policy in-process; inspect native task outcomes	3 seeds, 900 learned actions; 0/3 task successes; all reached the 300-step limit
Evaluation after fine-tuning	Evaluate the fine-tuned policy using the same reset seeds and episode limit	3/3 task successes; seeds 9201, 9202, and 9203; at most 300 steps per episode

Configuration. The source episode uses seed 4; its acceptance record includes task success, dataset validation, and a matching source hash. The agent kept the seven pose and gripper-opening fields of the recorded 13-dimensional vector (the other six are force measurements) and resized images to 224×224. Separate RGB and tactile ResNet-18 backbones, randomly initialized, feed an ACT Transformer with hidden dimension 128, four heads, two encoder and two decoder layers, and a 20-step action chunk. Training used all frames of the episode with batch size 4, learning rate 10^{-4}, seed 9173, no validation split, and a 200-optimization-step budget inside a 100-epoch limit; the eight recorded losses (every 26 steps) were finite, preparation plus training plus checkpoint checks took 29.8 s with 540 MiB peak GPU memory, and a checkpoint prediction returned a finite $1 \times 20 \times 7$ action tensor. During data inspection the gripper-opening assertion failed on 71 of 104 entries when actions were compared with observations at the same index; the corrected check compares each action with the following observation, with the source data and tolerance unchanged.

Execution. The source demonstrations use joint-position replay, while online execution uses an absolute differential IK controller in the robot base frame with the gripper joint range 0–0.04 m; the adapter converts axis-angle rotation to a quaternion and maps predicted opening into the gripper command range, using the normalization statistics saved with the checkpoint. Both evaluations used reset seeds 9201, 9202, and 9203 with a 300-step episode limit. The initial policy achieved

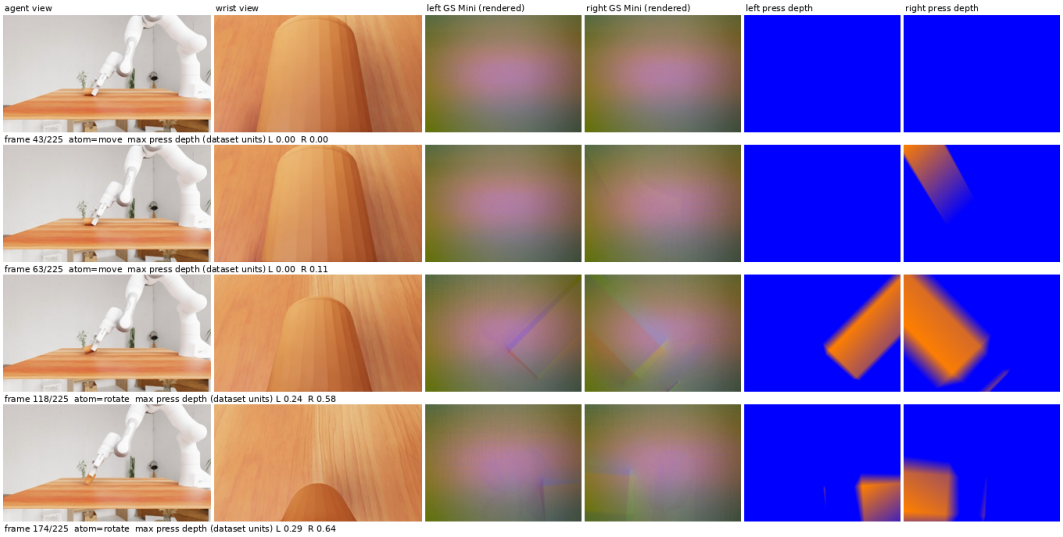


Figure 6: **Recorded tactile streams.** Four frames of a UniVTAC *lift can* source episode (file 000, 225 frames): agent and wrist RGB, the rendered left and right GelSight Mini images, and their press-depth maps (brighter is deeper). Row 1 precedes contact, row 2 is the first one-sided touch during the approach, and rows 3–4 are two-sided prints during the rotation phase; the per-frame maximum press depth is printed under each row in the dataset's units.

0/3 (episode runtimes 20.8, 20.5, and 20.4 s; 236.7 s including simulator startup) and the fine-tuned policy 3/3; predicted and executed actions, task signals, termination reasons, and episode videos are retained.

B TACTILE SIMULATION PIPELINE

Tactile streams come from two sensor models chosen by the environment adapter, not from a single renderer. The GelSight Mini pads used in the LIBERO/Tabero tasks and in UniVTAC follow the TacEx sensor model: Tabero renders the optical image with Taxim (Si & Yuan, 2021) and FOTS marker rendering from the PhysX contact depth, while UniVTAC simulates the gel with a UIPC soft body and renders it with a Taxim or pix2pix optical backend, returning RGB, marker, depth, and press-depth channels per pad. The DM-TacClaw pads used in the RoboLab, RLBench, LeHome, and IsaacLab replays use TacSim: a depth camera behind each authored pad surface measures indentation as baseline depth minus current depth, clipped at 1.5 mm; an object-filtered PhysX contact sensor gates every frame so that frames without physical contact are written as all-zero fields (the zero-contact contract); shear is the solenoidal and harmonic part of a Helmholtz decomposition of the surface deformation flow, whose irrotational part is attributed to normal indentation. Pad dimensions, camera placement, and gripper geometry follow the physical devices in Figure 4, and the GelSight Mini optical model uses the TacEx calibration. We do not claim photometric agreement with real sensor readings: no sim-to-real comparison of tactile images has been performed, and agent-side data inspection found a systematic under-reporting of the right TacClaw pad in the DM-TacClaw replays.

Figure 6 shows the four streams that the UniVTAC adapter delivers to training: two RGB views and the two rendered GelSight Mini images with their press-depth maps, taken from one of the 30 *lift can* source episodes used for TacJev. The pads report nothing before contact, a one-sided press when the gripper first touches the can, and two-sided prints during the rotation.